

GitHub über SSH nutzen

Contents

Git und SSH	1
Schlüssel generieren und ausgeben	1
Schlüssel zu GitHub hinzufügen	2
Schlüsselnutzung in SSH konfigurieren	6
Git über SSH nutzen	6
Weitere Informationen	7

Git und SSH

Git ist das am weitesten verbreitete Versionskontrollsystem (VCS) in der Softwarewelt. Wir benutzen Git auch für unsere Abgaben.

GitHub ist ein Service von Microsoft, der gratis Hosting von Git-Repositories anbietet. Es gibt immer wieder Probleme mit der Authentifizierung gegenüber GitHub. Das wird besser, wenn man SSH für die Anmeldung nutzt, anstatt einer HTTP-Verbindung mit Authentifizierung über API-Tokens.

SSH (**Secure SHell**) ermöglicht das Einloggen auf anderen Systemen über das Internet. OpenSSH ist die am weitesten verbreitete Implementation von SSH und seit Windows 10 (Update 1803) sogar [auf Microsoft-Systemen verfügbar](#). Linux, Mac und BSD (Unix-basierte Systeme) beinhalten SSH schon immer standardmäßig.

Schlüssel generieren und ausgeben

Um sich einzuloggen, muss¹ ein Schlüsselpaar generiert werden. Dieses besteht aus privatem und öffentlichem Schlüssel und ersetzt die Anmeldung via Passwort.

Die folgenden Befehle werden in einem Terminal ausgeführt, um ein Public/Private Schlüsselpaar zu generieren:

Info

Auf Windows:

Unter Windows sollte **PowerShell** (nicht `cmd`) genutzt werden. Hier kann es außerdem sein, dass wir ein `.exe` **Suffix** an das Kommando **anhängen** müssen, Sie also stattdessen `ssh-keygen.exe` ausführen. Dies ist beim `cat` Kommando nicht nötig.

Auf Linux/Mac/BSD:

Auf unixoiden Systemen sollte nichts Besonderes zu beachten sein, und die Befehle sollten in der standardmäßig vorinstallierten Shell^a unverändert funktionieren.

^aBash auf Linux, Zsh auf Mac, Ksh auf BSD.

¹"Muss" ist zwar strenggenommen falsch, aber da PubKey-Authentifizierung i.d.R. besser als die Authentifizierung mit Passwort ist und GitHub (AFAIK) die Nutzung von Schlüsseldateien wirklich verpflichtend vorgibt, tun wir hier einfach mal so, als wäre das eine generelle Regel.

```
# generiere schlüsselpaar mit ed25519 verschlüsselung
ssh-keygen -t ed25519
# gib den PUBLIC key auf dem terminal aus
cat ~/.ssh/id_ed25519.pub
```

Durch ssh-keygen wird ein Schlüsselpaar im .ssh Ordner in Ihrem Homeverzeichnis abgelegt (~, bzw. \$HOME unter Unix-Systemen und %userprofile% unter Windows). Den öffentlichen Schlüssel geben wir anschließend mittels cat (steht für concatenate) aus.

Schlüsselpaar

Das Schlüsselpaar wird genutzt, um sich bei irgendwelchen Diensten (wie z.B. GitHub) zu authentifizieren (anzumelden). Mithilfe des öffentlichen Schlüssels kann ein Dienst den Besitz des dazugehörigen privaten Schlüssels überprüfen werden, ohne dass der private Schlüssel dafür mit dem Dienst geteilt werden müsste.

Der **ÖFFENTLICHE** Schlüssel sieht ungefähr so aus:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFbohldIVjN0hcRpOngCMuGbSwedulQXgw0erzfUfioE fynn@laptop
```

Dieser darf weitergegeben werden.

Der dazugehörige **PRIVATE** Schlüssel sieht z.B. so aus:

```
-----BEGIN OPENSSH PRIVATE KEY-----
b3B1bnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMwAAAAAtzc2gtZW
QyNTUxOQAAACBW6IZXSFYzdIXEaTp4AjLhm0sHnbpUF4MDnq831H4qBAAAAJD1sH0H5bB9
BwAAAAAtzc2gtZWQyNTUxOQAAACBW6IZXSFYzdIXEaTp4AjLhm0sHnbpUF4MDnq831H4qBA
AAAEaTo+j3bmJyjiH3T26WG11LqpOnZh22hFnFlG0C+rb1bohldIVjN0hcRpOngCMuGb
SwedulQXgw0erzfUfioEAAAAC2Z5bm5AbGFwdG9wAQI=
-----END OPENSSH PRIVATE KEY-----
```

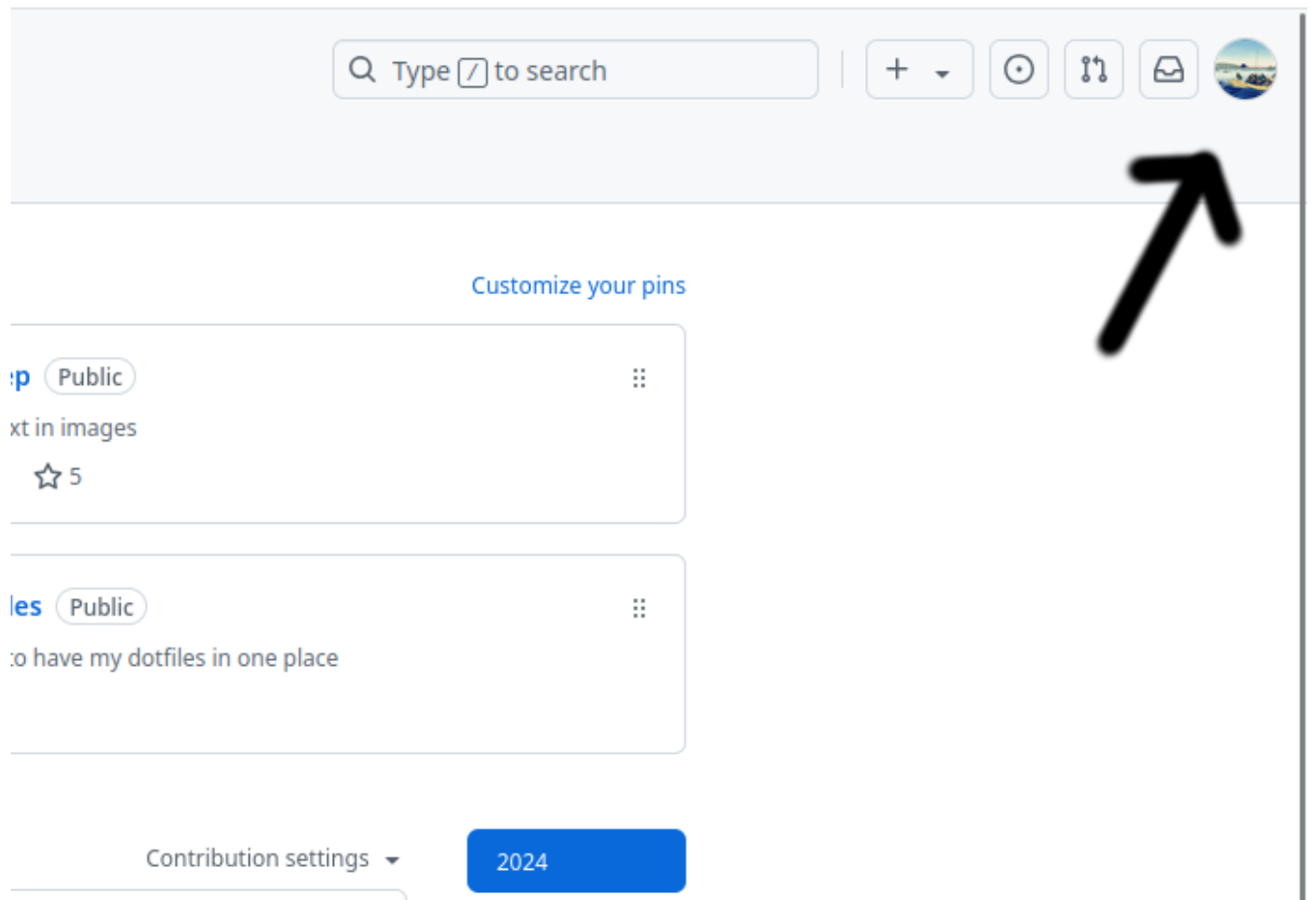
Danger

Der private Schlüssel wird NIEMALS geteilt. Er ist quasi euer Passwort und wird niemandem verraten. Wenn Tech-Support euch nach eurem Private-Key fragt, dann ist es ein Scam und ihr legt bitte sofort auf.

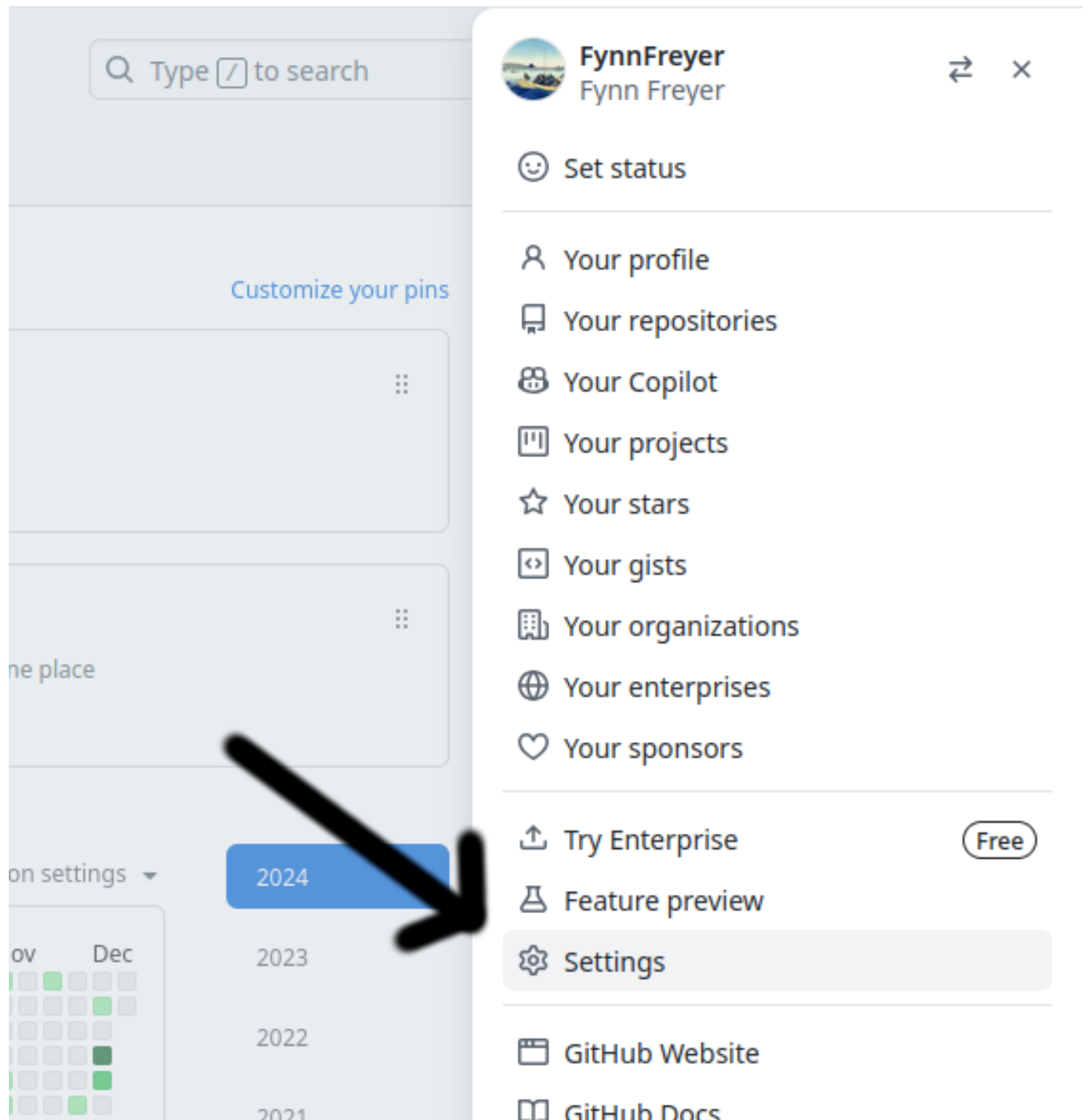
Der öffentliche Schlüssel dagegen kann gefahrlos weitergegeben werden.

Schlüssel zu GitHub hinzufügen

Um den öffentlichen Schlüssel in GitHub zu hinterlegen, geht ihr wie in der [GitHub-Dokumentation](#) beschrieben vor. Hier eine Kurzanleitung mit Bildern:



1. Öffne das Seitenmenü über das Profilanzeigebild oben links.



2. Öffne die Einstellungen.



Fynn Freyer (FynnFreyer)

Your personal account [Switch settings context](#)

Public profile

- Account
- Appearance
- Accessibility
- Notifications

Access

- Billing and plans
- Emails
- Password and authentication
- Sessions
- SSH and GPG keys**
- Organizations
- Enterprises
- Moderation

Code, planning, and automation

Repositories

- Public profile
- Account
- Appearance
- Accessibility
- Notifications

Access

- Billing and plans
- Emails
- Password and authentication
- Sessions
- SSH and GPG keys**
- Organizations
- Enterprises
- Moderation

Code, planning, and automation

- Repositories
- Codespaces

Public profile

Name

Fynn Freyer

Your name may appear around GitHub where you contribute or are mentioned.

Public email

fynn.freyer@googlemail.com [Remove](#)

You can manage verified email addresses in your [email settings](#)

Bio

I like trains

You can @mention other users and organizations to link to them.

Pronouns

Don't specify

URL

https://fynns.site

3. Öffne den SSH und GPG Reiter.

SSH keys

This is a list of SSH keys associated with your account. Remove any keys that you do not recognize.

Authentication keys

**id_github**
SHA256:m5q8kBIoXMDpFPEpV7vqBySoXA4sM57c/sUPYUvwKvA
Added on Apr 11, 2023
Last used within the last 2 weeks — Read/write
[Delete](#)

Check out our guide to [connecting to GitHub using SSH keys](#) or troubleshoot [common SSH problems](#).

GPG keys

There are no GPG keys associated with your account.

Learn how to [generate a GPG key and add it to your account](#).

Vigilant mode

☐ Flag unsigned commits as unverified

This will include any commit attributed to your account but not signed with your GPG or S/MIME key. Note that this will include your existing unsigned commits.

4. Füge einen neuen Schlüssel hinzu.

Add new SSH Key

Title

New Key

Key type

Authentication Key ↕

Key

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIFbohldIVjN0hcRpOnqCMuGbSwedulQXgwOerzfUfioE fyinn@laptop|

Add SSH key



5. Füge den Public Key hinzu und Bestätige den neuen Schlüssel.

Schlüsselnutzung in SSH konfigurieren

Angenommen der Schlüssel liegt unter `~/.ssh/id_ed25519`, dann kann SSH so konfiguriert werden, dass dieser Schlüssel automatisch für `github.com` verwendet wird. Dafür muss Folgendes in `~/.ssh/config` hinterlegt werden:

```
Host github.com
  IdentityFile ~/.ssh/id_ed25519
```

Git über SSH nutzen

Um nun Git über SSH zu nutzen betrachten wir oberflächlich zwei verschiedene Szenarien. Bei Problemen oder Detailfragen können Sie gerne in der Übung auf mich zukommen.

Neue Repositories klonen

In Zukunft wird zum Hinzufügen eines Repositorys die URL aus dem SSH-Reiter verwendet.

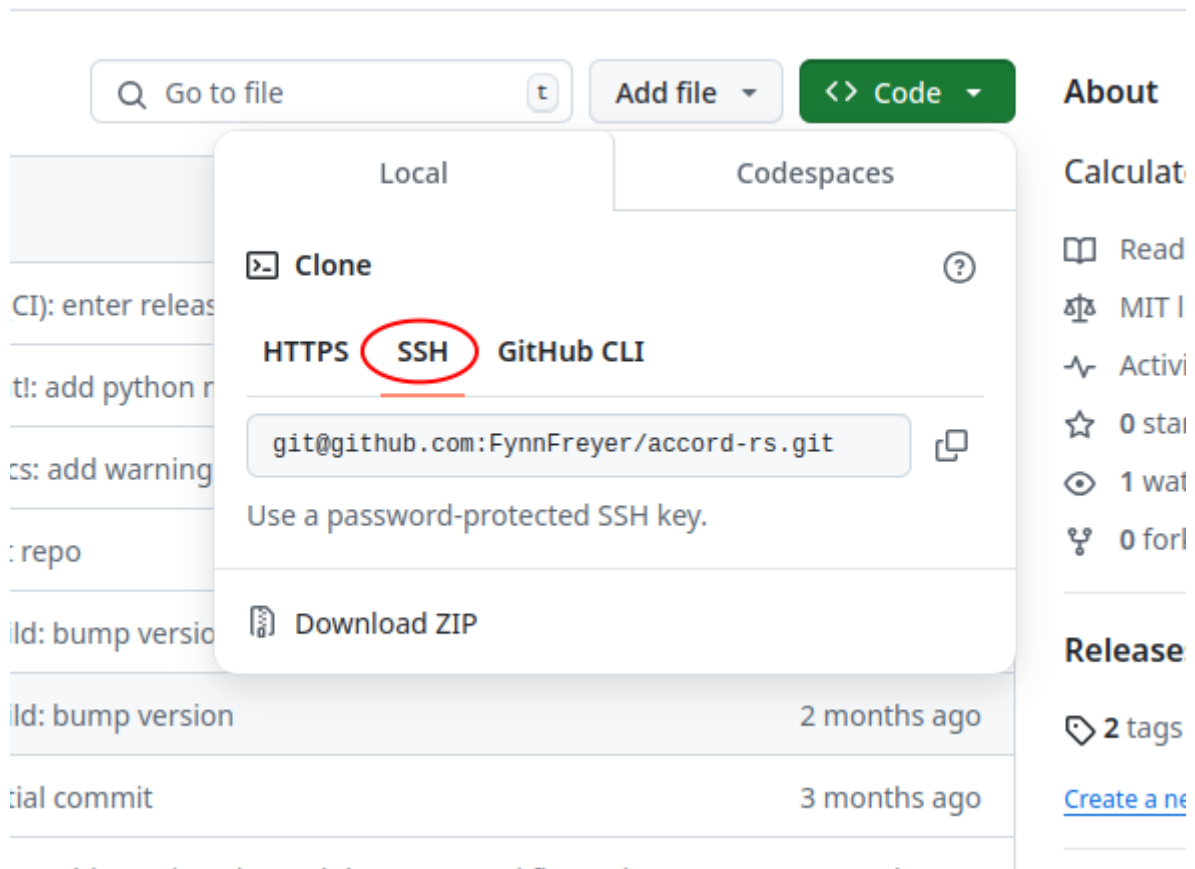


Figure 1: Der SSH Reiter

Diese Adresse könnt Ihr sowohl in eurer IDE verwenden (z.B. unter "Clone Repository" oder "Project from Version Control ...") oder in der Kommandozeile (mit `git clone git@github.com:User/Repo.git`).

Alte Repositories auf SSH umstellen

Auch bereits geklonte Repositories können mit den folgenden Befehlen auf SSH umgestellt werden:

```
# delete the old remote
git remote rm origin
# add a new remote. "User" and "Repo" need to be changed accordingly
git remote add origin git@github.com:User/Repo.git
```

Hier wird der alte HTTP-Remote entfernt und ein neuer SSH-Remote hinzugefügt, auf den ab sofort gepusht werden kann.

Weitere Informationen

Mehr zu Git und SSH finden Sie im Internet. Z.B. diese Cheat-Sheets [für Git](#) und [für SSH](#).